SOLANA STABLECOIN

PAYMENT INFRASTRUCTURE

Architecture Design & Flow Diagrams

Production Reference — v1.1

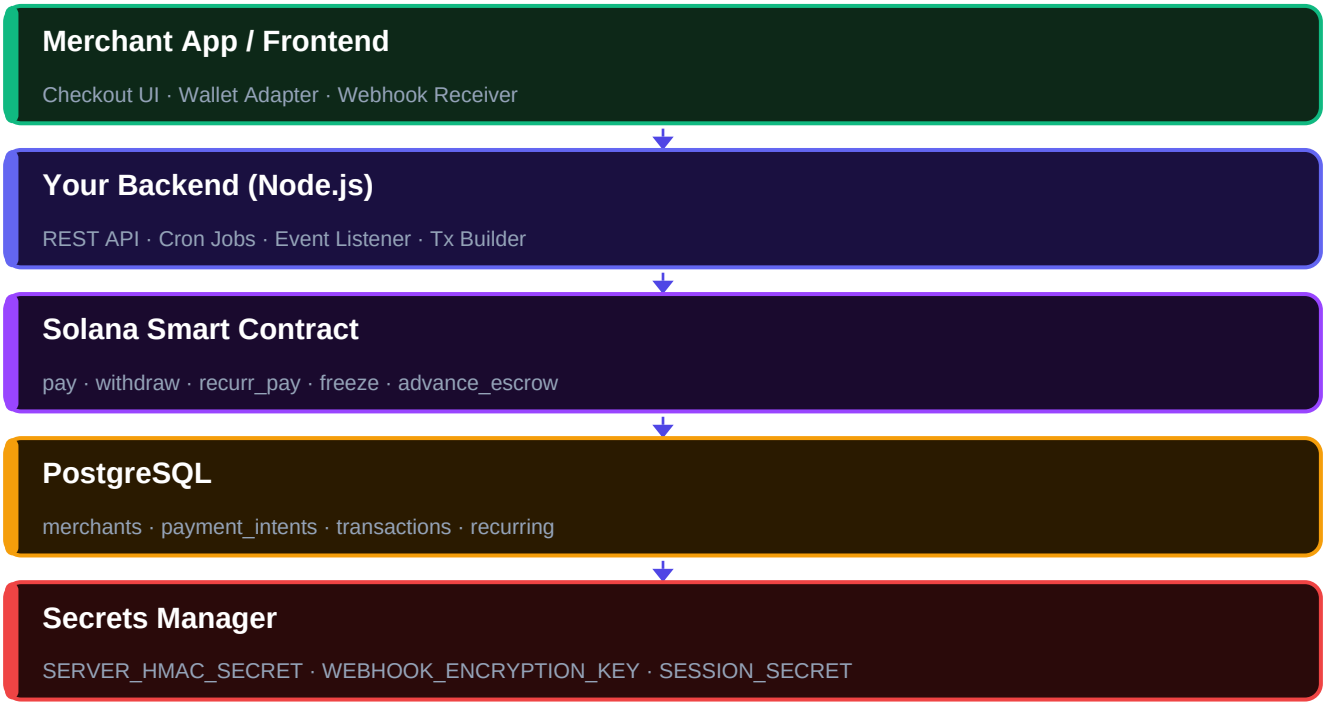
Layer	Technology	Responsibility
Merchant App	React / Next.js + Wallet Adapter	Checkout UI, Webhook receiver
Backend API	Node.js / TypeScript	REST API, Cron jobs, Tx builder
Smart Contract	Solana / Anchor (Rust)	All fund movement, vault state
Database	PostgreSQL	Payment intents, tx log, merchants
Secrets	AWS KMS / HashiCorp Vault	HMAC keys, encryption keys, JWT secret

Property	Value
Fee model	2% per transaction (platform_fee_bps = 200)
Escrow period	7 days via on-chain circular buffer
Stablecoin	USDC (extensible to other SPL tokens)
Backend authority	NONE — contract controls all fund movement
Recurring billing	Delegate-and-pull via RecurringPDA + daily cron
Min superadmins	2 (enforced on-chain in create_admin)

Chapter 1 — System Architecture Overview

A five-layer stack where the backend never holds authority over funds. The Solana contract is the single source of truth for all token movement.

1.1 Five-Layer Stack



The key design constraint: a compromised backend cannot steal funds. The contract is the only entity that moves tokens.

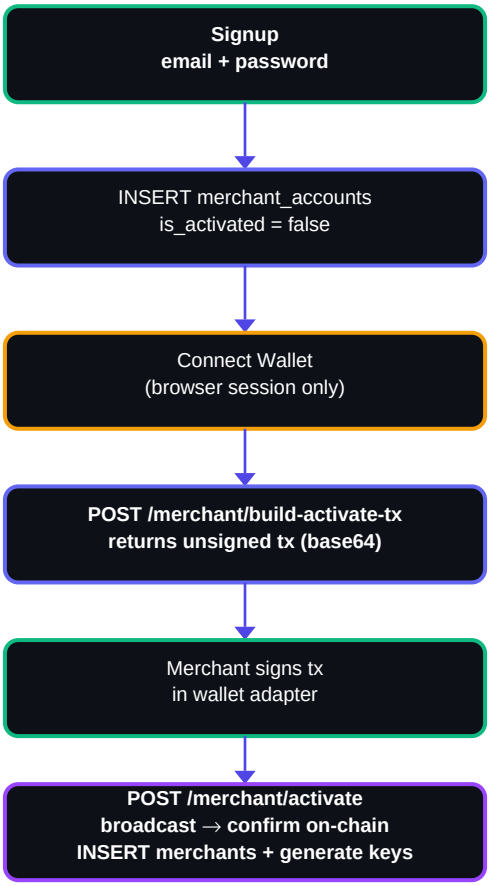
1.2 Core Design Constraints

Constraint	Rationale
Backend never holds/moves funds	Compromised backend key = 0 fund risk
Contract is source of truth for amounts	No DB amount can be passed to contract — on-chain math only
Two API key types (sk_ / pk_)	Frontend-safe publishable key for checkout; secret key for backend only
Blockhash refreshed at signing time	Prevents expired-blockhash tx rejection after 90s checkout sessions
Signed tx validated server-side before broadcast	Blocks malicious vault-swap or amount-change attacks
Webhook secret encrypted (AES-256-GCM)	DB breach alone cannot expose webhook secrets
freeze duration-capped (30 days max)	Prevents indefinite vault lockout by compromised operator key

Chapter 2 — Merchant Registration Flow

Two-stage onboarding separates credential creation (off-chain) from vault activation (on-chain). API keys are only generated after on-chain confirmation — never before.

2.1 Two-Stage Activation Flow



Stage 1: Email/password → merchant_accounts row (is_activated=false). Stage 2: Wallet sign → on-chain confirm → merchants row + API keys generated once.

2.2 Database Tables

```
merchant_accounts: id, email, password_hash (bcrypt), is_activated, created_at
merchants: id, merchant_account_id (FK, UNIQUE), wallet_pubkey, secret_key_id,
secret_key_hash, publishable_key_id, publishable_key_hash,
webhook_url, webhook_secret (AES-256-GCM ciphertext), created_at
```

2.3 Critical Rules

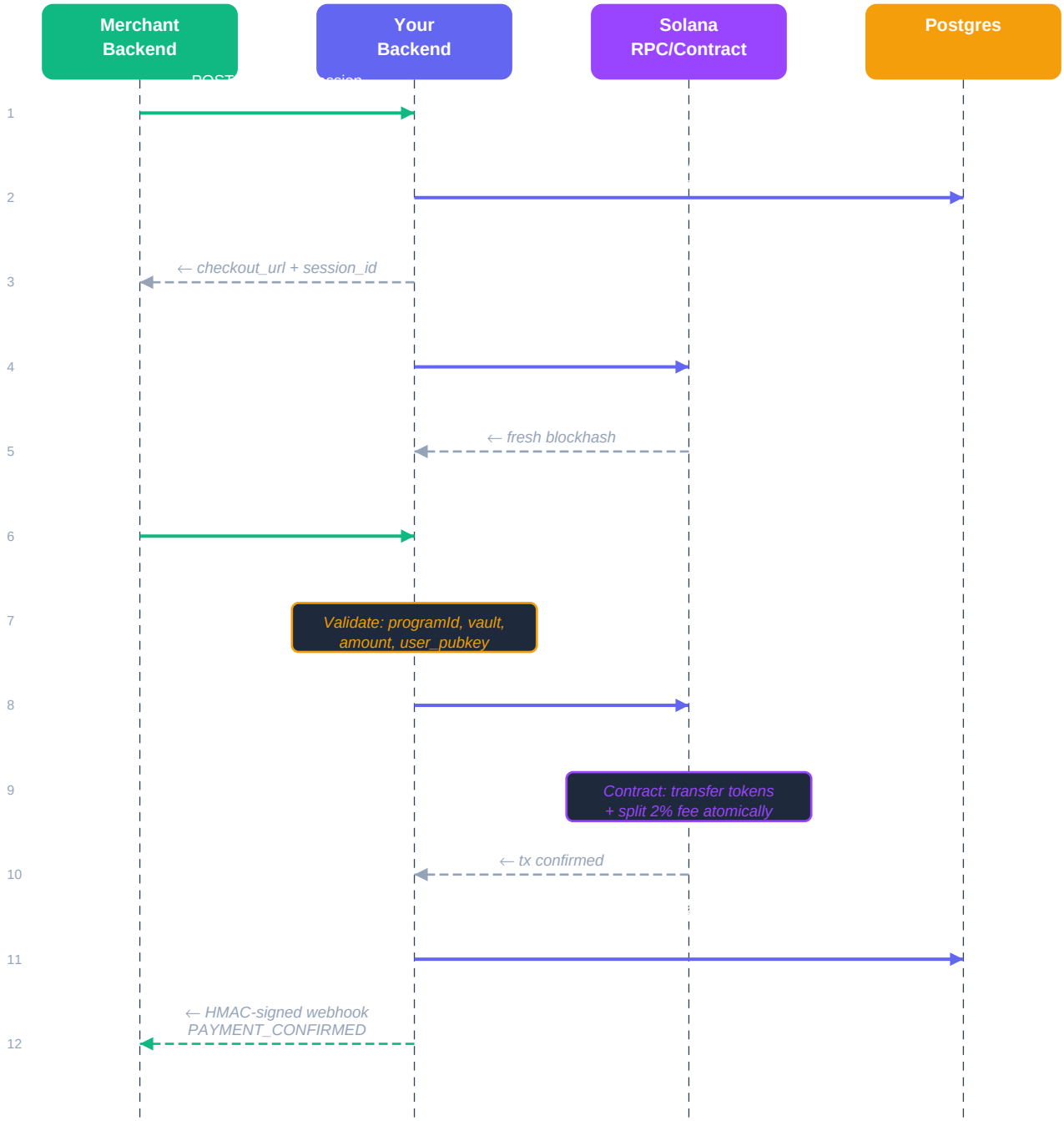
- wallet_pubkey is NEVER persisted until on-chain tx is confirmed — merchant may swap wallets before signing
- API keys are generated ONLY after confirmation — failed tx = no merchants row, no keys
- webhook_secret stored as AES-256-GCM ciphertext (not bcrypt) — must be decryptable for signing
- is_activated = true set atomically with merchants INSERT — retry-safe via 409 on duplicate

■ *Why AES-256-GCM instead of bcrypt for webhook_secret? bcrypt is one-way. You need the raw secret to sign outbound payloads — you cannot sign with a hash. Encrypt with WEBHOOK_ENCRYPTION_KEY from secrets manager; decrypt at dispatch time only.*

Chapter 3 — One-Time Payment Flow

9-step sequence from checkout session creation through webhook delivery. Every step has an exact reason — deviating introduces a security gap or state inconsistency.

3.1 Sequence Diagram



Solid arrows = requests. Dashed arrows = responses. Orange boxes = server-side validation steps.

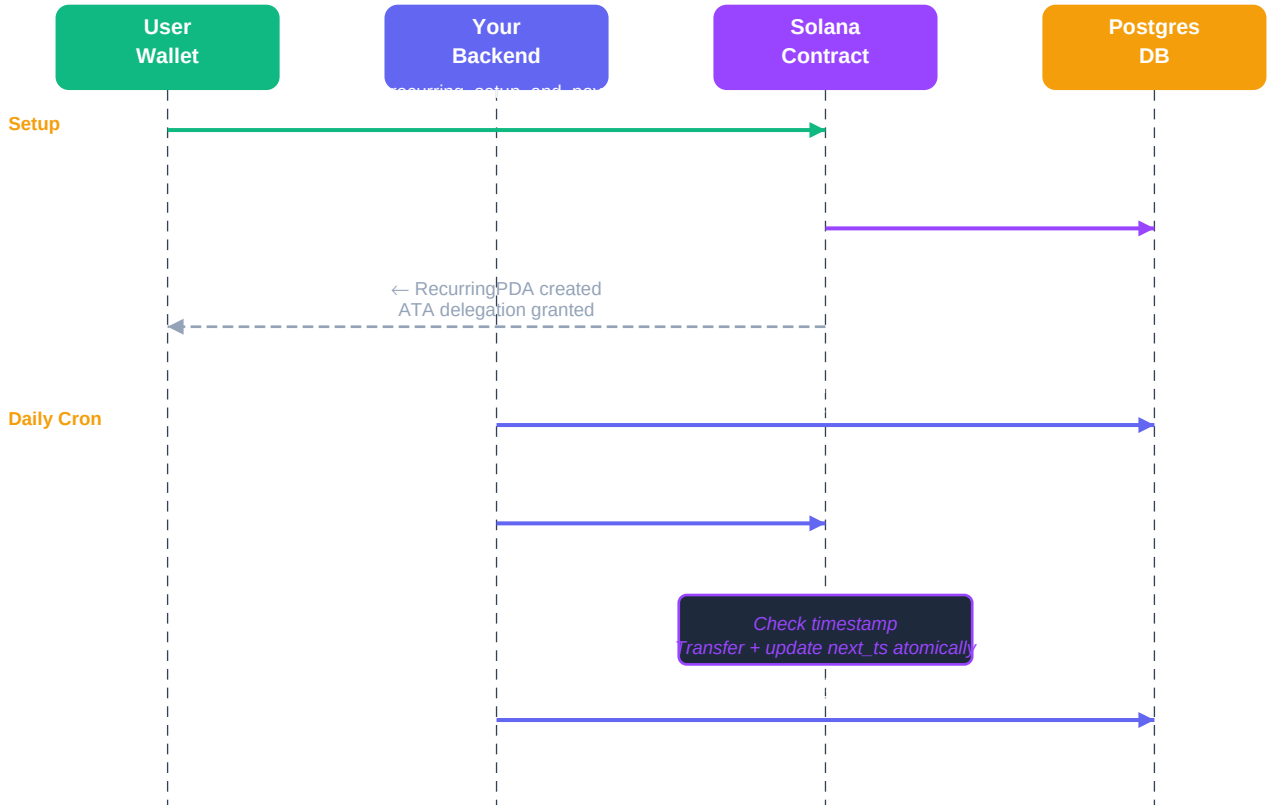
3.2 Step-by-Step Reference

#	Actor	Action	Critical Detail
1	Merchant BE	POST /checkout/session	idempotency_key = merchant_id:order_id:amount — INSERT ON CONFLICT DO NOTHING
2	Your BE	Create PaymentIntent	status='pending', expires_at = now() + 10min
3	Checkout page	Fetch fresh blockhash	NEVER reuse blockhash from step 1 — expires after ~90s
4	Customer	Sign tx in wallet	Wallet signs the tx with fresh blockhash
5	Your BE	Validate signed tx	Check: programId, vault address, amount, user_pubkey vs PaymentIntent
6	Your BE	sendTransaction()	Broadcast to RPC — do NOT broadcast without step 5 validation
7	Contract	Transfer tokens	pay(): (amount-fee) → vault, fee → admin_fee_vault, atomically
8	WS Listener	PaymentReceived event	INSERT transactions, DELETE PaymentIntent, enqueue webhook
9	Reconciliation	5-min cron fallback	Cursor-based scan catches any events WS listener missed

Chapter 4 — Recurring Payment Flow

Delegate-and-pull pattern: user grants ATA delegation to the program once. Daily cron executes recurr_pay atomically — timestamp update and transfer in same instruction.

4.1 Recurring Sequence



PDA existence = active subscription. PDA closed (via stop_recurr_pay) = cancelled. No active flag needed.

4.2 RecurringPDA Seed Design

```
seeds: ["recurring", user_pubkey, merchant_pubkey, subscription_id]
```

- subscription_id in seeds — cancel + resubscribe increments ID, creates new PDA at new address
- Original flaw: seeds had no subscription_id → cancel + resubscribe impossible (closed PDA address)
- next_payment_timestamp updated atomically with transfer — prevents double-charge on concurrent cron workers

4.3 Cancellation (stop_recurr_pay)

- signer must be recurring_pda.user — merchants cannot cancel user subscriptions
- Closes RecurringPDA via Anchor close = user → rent returned to user
- Conditionally revokes ATA delegation — ONLY if no other RecurringPDAs exist for same user+merchant
- Do NOT revoke if user has other active subscriptions with same merchant

Chapter 5 — Escrow & Withdrawal Architecture

Funds are held 7 days in an on-chain circular buffer before becoming withdrawable. The contract is the sole source of truth — the cron passes no amount.

5.1 7-Slot Circular Buffer

7-Slot Circular Buffer (withheld_buckets[0..6])

Daily cron advances current_index → oldest slot credited to withdrawable_amount. Missed days auto-recover via advance_escrow().

5.2 MerchantPDA State

```
total_amount = withheld_amount + withdrawable_amount (INVARIANT — always enforced)
withheld_buckets : [u64; 7] — circular day-bucket write buffer
current_index : u8 — write position (0-6)
current_date : i64 — unix ts of last advance
withdrawable_amount — merchant can withdraw from this
```

5.3 advance_escrow() Logic

Condition	Action
diff == 0 (same day)	No-op — AlreadyUpdatedToday error if cron calls twice
diff 1–6 (partial)	Iterate each missed slot: add to withdrawable_amount, zero slot
diff >= 7 (full drain)	Drain entire buffer → withdrawable_amount, reset current_index = 0
Called from pay/withdraw	Auto-catches up if cron was missed — no separate recovery needed

5.4 Withdrawal Guards (withdraw instruction)

- Auto-expiry: if freeze_flag && Clock::get() > freeze_expires_at → clear freeze fields, proceed
- require!(!freeze_flag) — vault must not be frozen
- require!(transfer_flag == true) — superadmin must enable withdrawals
- require!(merchant_ata.owner == merchant_signer) — only merchant can withdraw
- require!(withdraw_amount <= withdrawable_amount) — cannot withdraw held funds
- advance_escrow() called first — catches up any missed days before balance check

Chapter 6 — Security Architecture

6.1 API Key Architecture

`sk_live_{keyId}_{secret}`

All backend routes
Full account access

Risk: HIGH — backend only

`pk_live_{keyId}_{secret}`

POST /checkout/session
only

Risk: LOW — safe for frontend

Auth Middleware Flow:

Extract keyId from x-api-key header → O(1) SELECT WHERE secret_key_id = keyId → HMAC-SHA256(secret, SERVER_HMAC_SECRET) → timingSafeEqual(computed, stored_hash) → Route scope check → 401 / 429 / continue

6.2 Key Format & Storage

```
Format: {prefix}_{env}_{keyId}_{secret}
Example: sk_live_abc123_
```

- keyId = indexed lookup handle (not secret) — enables O(1) DB lookup
- secret portion verified with HMAC-SHA256(secret, SERVER_HMAC_SECRET)
- Stored in DB: keyId (indexed) + HMAC hash of secret — raw secret never stored
- Auth: timingSafeEqual() always — never == or === (timing attack prevention)
- Rate limit: 100 req/min per API key → 429 with Retry-After header

6.3 Secrets Manager Reference

Secret	Purpose	Rotation Impact
`SERVER_HMAC_SECRET`	Hash API key secret portions	Rotate = invalidate all API keys
`WEBHOOK_ENCRYPTION_KEY`	AES-256-GCM for webhook secrets	Rotate = re-encrypt all webhook secrets
`SESSION_SECRET`	Sign merchant JWTs	Rotate = invalidate all active sessions
`BACKEND_PRIVATE_KEY`	Solana keypair for cron instructions	Rotate keypair + update operators list

6.4 Transaction Validation (most critical security step)

Before calling sendTransaction on ANY signed tx from client, validate ALL of:

Field	Check Against	Attack Prevented
`program ID`	Deployed contract address	Malicious contract substitution
`merchant vault account`	Merchant's actual on-chain PDA vault	Vault address swap → steal funds

`transfer amount`	PaymentIntent.amount exactly	Amount reduction to 1 token unit
`user pubkey`	PaymentIntent.user_pubkey	Identity substitution
`recentBlockhash`	Valid via RPC check	Expired tx replay
`PaymentIntent status`	Must be 'pending'	Double-submit race condition

Chapter 7 — Cron Jobs & Failure Handling

7.1 Cron Job Reference

Cron Job	Schedule	What It Does	Idempotent?
Reconciliation	Every 5 min	Cursor-based scan via getSignaturesForAddress — inserts missed txs, unique constraint prev	Yes — UNIQUE constraint prevents dupes
Escrow Advance	Daily	Invokes update_withdrawal_amount per merchant — contract enforces once-per-day via Alre	Yes — contract rejects AlreadyUpdatedToday
Recurring Payments	Daily	SELECT WHERE next_ts <= NOW() → recurr_pay per row — on PDA-not-found, delete DB r	Yes — PDA check + delete

7.2 Failure Handling Matrix

Failure	Impact	Recovery
WS listener disconnects	Missed PaymentReceived events	Exponential backoff reconnect (1s→30s max). 5-min cron catches all within one cycle.
DB write after tx confirmed	Tx confirmed, not recorded	UNIQUE on tx_signature prevents dupes. Reconciliation cron picks up within 5 min.
Network timeout on broadcast	Unknown tx status	Never mark failed on timeout. Recon cron checks chain. Mark failed only if confirmed-no
Escrow cron runs twice	Duplicate update attempt	Contract rejects with AlreadyUpdatedToday. Log and skip — fully idempotent.
Vault frozen	Merchant cannot withdraw	freeze_expires_at auto-expires in withdraw instruction. Max 30 days on-chain enforced.
Activation tx broadcast fails	No on-chain vault created	is_activated stays false, no merchants row. Merchant retries from clean state.
Activation tx confirmed, DB write fails	Vault exists, no keys	Log tx_signature. Retry detects on-chain confirmation idempotently before reprocessing.
recurr_pay — insufficient balance	Subscription missed	Set last_failure_reason. Enqueue PAYMENT_FAILED webhook to merchant.
Blockhash expired on submit	Tx rejected by network	Return error to checkout page. User restarts. Checkout ALWAYS refreshes blockhash b

Chapter 8 — Database Schema Reference

merchant_accounts

Column	Type / Constraint	Notes
id	UUID PK	gen_random_uuid()
email	TEXT UNIQUE NOT NULL	
password_hash	TEXT NOT NULL	bcrypt saltRounds=10
is_activated	BOOLEAN DEFAULT false	
created_at	TIMESTAMPTZ	NOW()

merchants

Column	Type / Constraint	Notes
id	UUID PK	
merchant_account_id	UUID UNIQUE FK	→ merchant_accounts.id
wallet_pubkey	TEXT UNIQUE NOT NULL	
secret_key_id	TEXT UNIQUE	O(1) lookup handle
secret_key_hash	TEXT	HMAC-SHA256 of secret portion
publishable_key_id	TEXT UNIQUE	
publishable_key_hash	TEXT	
webhook_url	TEXT	
webhook_secret	TEXT	AES-256-GCM ciphertext

payment_intents

Column	Type / Constraint	Notes
id	UUID PK	
merchant_id	UUID FK	→ merchants.id
order_id	TEXT	
amount	BIGINT	
user_pubkey	TEXT	
idempotency_key	TEXT UNIQUE	merchant_id:order_id:amount
status	TEXT DEFAULT 'pending'	
expires_at	TIMESTAMPTZ	now() + 10 min

transactions

Column	Type / Constraint	Notes
id	UUID PK	
merchant_id	UUID FK	→ merchants.id
tx_signature	TEXT UNIQUE	prevents duplicate inserts
amount	BIGINT	
user_pubkey	TEXT	

order_id	TEXT	
created_at	TIMESTAMPTZ	

recurring

Column	Type / Constraint	Notes
id	UUID PK	
merchant_id	UUID FK	→ merchants.id
user_pubkey	TEXT	
subscription_id	BIGINT	mirrors PDA seed
amount	BIGINT	
next_payment_timestamp	BIGINT	mirrors RecurringPDA
last_failure_reason	TEXT	row existence = active

Chapter 9 — Smart Contract Instruction Reference

`create_admin`	Requires min 2 superadmins (enforced on-chain). MAX_SUPERADMINS=3, MAX_OPERATORS=10.
`create_merchant`	Creates MerchantPDA + merchant vault ATA atomically. transfer_flag=false, freeze_flag=false. Merchant wallet is sig
`pay`	Transfers (amount-fee) → merchant vault + fee → admin_fee_vault atomically. Calls advance_escrow() first. Min fee
`withdraw`	Auto-expires freeze first. Checks transfer_flag, freeze_flag, merchant_ata.owner. advance_escrow() before balance c
`update_withdrawal_amount`	Called by escrow cron daily. No amount arg — math fully on-chain. Enforces once-per-day via AlreadyUpdatedToday
`freeze_vault`	Callable by operators OR superadmins. Takes duration_seconds (max 30 days). Auto-expires — no separate cron n
`recurr_pay`	PDA existence = active check. Timestamp check first. Transfer + next_payment_timestamp update ATOMICALLY. P
`stop_recurr_pay`	Only user can cancel (signer == recurring_pda.user). Closes PDA (rent returned). Conditionally revokes ATA delegat
`remove_superadmin`	require!(superadmins.len() > 1) — cannot remove last superadmin. Only callable by superadmins.

9.1 Contract Events

Event	Instruction	Key Fields
`PaymentReceived`	pay	merchant, user, amount, fee, timestamp
`EscrowAdvanced`	update_withdrawal_amount	merchant, slots_advanced, amount_released, new_withdrawable, timesta
`WithdrawExecuted`	withdraw	merchant, amount, timestamp
`VaultFrozen`	freeze_vault	merchant, frozen_by, expires_at, timestamp
`VaultUnfrozen`	unfreeze_vault	merchant, unfrozen_by, timestamp
`RecurringSetup`	recurring_setup_and_pay	user, merchant, amount, next_payment_at
`RecurringPulled`	recurr_pay	user, merchant, amount, next_payment_at
`RecurringStopped`	stop_recurr_pay	user, merchant, stopped_at